



UPPSALA
UNIVERSITET

Introduction to Literature Seminar and Mini-Projects



UPPSALA
UNIVERSITET

Literature seminar and mini-project are group actives

Group names: **DE-II 1**

DE-II 2

DE-II 19

Group size: **5 or 4 students**



UPPSALA
UNIVERSITET

Literature Seminar

- This task is to be completed with your group, **deadline: 13-05-2026**
- The literature seminar consists of two parts:
 - Preparing by reading scientific papers and writing answers to the provided questions **based on the group size**. Aim for about half A4 page per question using a font-size of 12pt. Also, for each paper, prepare one possible question for discussion
 - Active participation in the seminar, where we will discuss and share our understanding of the material
- If you for some reason cannot participate in the seminar, you should contact the teacher to get a **make-up assignment**
- In the Studium, you will find scientific papers that are relevant to the course, and that we think will be helpful references to complete the written part of the task



UPPSALA
UNIVERSITET

Literature Seminar

- M1 - Contextualization and Containers
- M2 - Data Stream Processing
 - *Apache Spark Streaming, Kafka and HarmonicIO: A Performance Benchmark and Architecture Comparison for Enterprise and Scientific Computing*
 - *HarmonicIO: Scalable Data Stream Processing for Scientific Datasets*
 - ...
- M3 - Orchestration frameworks and production-grade computational workflows
 - *Batch and online anomaly detection for scientific applications in a Kubernetes environment*
 - *Architectural Implications of Function-as-a-Service Computing*
 - *Towards a Contextualization Solution for Cloud Platform Services*
 - *Contextualization: dynamic configuration of virtual machines*
 -
- M4 - Federated and distributed machine learning
 - *A Survey on Distributed Machine Learning*
 - *From distributed machine learning to federated learning: In the view of data privacy and security*
 -



UPPSALA
UNIVERSITET

- Collaboration document!
- Compulsory to submit with the report

Literature Seminar

Group Contribution Document

Project:

Date: DD-MM-YYYY

Group name: AAC-100

Group members: AAA, BBB, CCC, DDD

Contributions

AAA: Read two articles "Article-1" and "Article-2", contributed on the framework design, work on overall framework integration and contribute in the report writing.

BBB: Implement first component of the framework, contributed on the framework design and participated in the report writing.

CCC: Implement third component of the framework, test the working of the framework, contribute in the report writing.

DDD: Read three articles "Article-1", "Article-2" and "Article-3", Implement the second component and contributed in report writing.

Signature and Date:

AAA

BBB

CCC

DDD



Projects

- **Project 1**
 - Title: Demonstration of horizontal scaling with lightweight virtualization approach for large-scale unit test analysis of open-source projects
- **Project 2**
 - Title: Github analytic system using a streaming framework
- **Project 3**
 - Title: Evaluating the accuracy of prediction of stargazers in open source projects



UPPSALA
UNIVERSITET

Project-1: Demonstration of horizontal scaling with lightweight virtualization approach for large-scale unit test analysis of open-source projects

Key words: Distributed Computing Infrastructures, Scalability, Orchestration, software deployment



Project-1

- Analyzing large-scale open-source projects is an active research that allows researchers to understand real-world projects and development issues
- Working with the corpus of code is a very time-consuming process that can lead to high wait time, especially if the analysis is done at runtime.
- **Task:** Develop a framework for optimizing large scale open source project analysis with container orchestration techniques



Project-1

- You will use Github repositories for the project-1



- Git is an open-source version control system that was started by Linus Torvalds—the same person who created Linux
- Version control systems keep revisions straight, storing the modifications in a central repository.
- Allows developers to easily collaborate, as they can download a new version of the software, make changes, and upload the newest revision.
- Every developer can see these new changes, download them, and contribute.



Different between Git and GitHub

- Git is a command-line tool
- GitHub is a platform where developers store their projects and network with like minded people
- Version control systems are not only for developers!



UPPSALA
UNIVERSITET



Project-1

- To start with the project, first you need to write a **project crawler**
- A crawler, spider, or search engine is a software module that downloads and indexes content from the internet.
- Within the scope of this project, *project crawler* will collect information from the GitHub repositories.
- To write a *project crawler* you need to send query to GitHub using GitHub search API.



Project-1

- Information about GitHub API, such as using the API, authentication, and core concepts, is available on <https://docs.github.com/en/rest>
- GitHub API returns project information that needs to be downloaded to perform code analysis
- Since manually running each open source project is a cumbersome task, we will leverage unit tests of the project and build-system that can allow automatic execution of projects
- In this regard, we will only select **1000 Java projects** having a Maven build system



Project-1

- Tasks:
 - Design a scalable architecture for the framework
 - Develop a complete software solution based on your proposed architecture
- You can design different experiments to showcase the platform's scalability and performance. Following are the recommended scalability tests:
 - Strong scalability
 - Weak scalability
- The implemented solution should have following features:
 - Automation
 - Contextualization
 - Orchestration



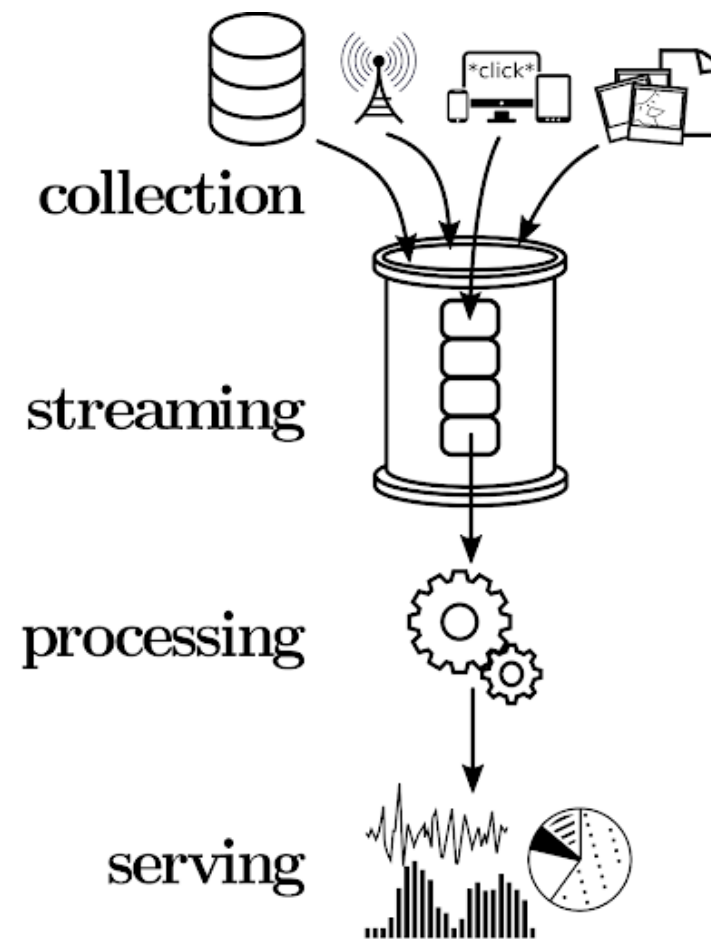
UPPSALA
UNIVERSITET

Project 2: Github analytic system using a streaming framework

Key words: Stream Processing, Scalability, Orchestration,
software deployment



UPPSALA
UNIVERSITET



Project-2

- GitHub is one of the leading public repository services, consisting of many open source projects
- The provided services by Git do not offer a ready to use solution for in-depth information about projects
- **Task:** Develop an analytic system for GitHub



Project-2

- For this project you need to use a streaming framework for processing repositories metadata
- One can use any streaming framework. However, if you need to discuss your streaming solution, the recommendation is to use Apache Pulsar.
- In this project, the focus is to analyze the repositories **metadata**
- The aim is to stream data from GitHub by using GitHub API to answer several key questions that would not be possible with existing GitHub search



Project-2

- Metadata information regarding open source projects can be collected with GitHub API
- For this project, we will limit our criteria to the last year
- Challenges:
 - **Search Limit:** GitHub API only returns a maximum of 1000 searches per date
 - **Rate Limit:** Most Rest APIs limit the query with a rate limit
 - **Analytics:** A streaming framework processes repository metadata information



Project-2

- The analytic system should be able to answer the following questions about repositories on GitHub.
 - **Q1.** What are the top 10 programming languages based on the number of projects developed?
 - **Q2.** What are the top 10 most frequently updated GitHub projects (i.e., most commits in a project)?
 - **Q3.** What are the top 10 programming languages that follow the test-driven development approach (i.e., most projects with unit tests)?
 - **Q4.** What are the top 10 programming languages that follow test-driven development and DevOps approach (i.e., uses continuous integration in the development)?



Project-2

- Tasks:
 - Design a scalable architecture for the stream processing framework
 - Develop a complete software solution based on your proposed architecture
- You can design different experiments to showcase the framework's efficiency and performance. Following are the recommended experiments:
 - Adaptive query design
 - Analysis based on topics and partitions
- The implemented solution should have following features:
 - Automation
 - Contextualization
 - Orchestration



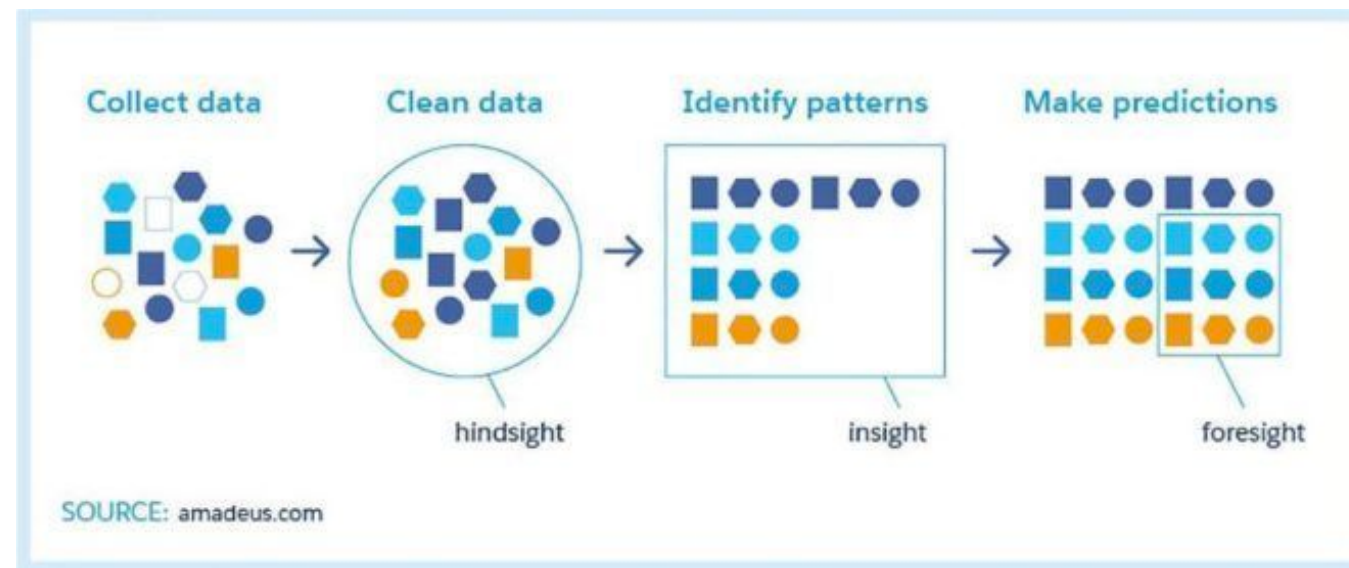
UPPSALA
UNIVERSITET

Project 3: Evaluating the accuracy of prediction of stargazers in open source projects

Key words: Predictive analysis, Scalability, Orchestration, Software deployment



Project-3



- Online platforms use popularity as a measure to identify who is performing better than the others
- *YouTube* subscriber information can be used as a direct measure for a channel's popularity
- *GitHub* users show their appreciation for a project by putting stargazer (or star) to the project
- The number of stargazers indicate the popularity of a GitHub project



Project-3

- **Task:**

- To study which prediction model has the highest accuracy in predicting the number of stars for a GitHub repository.

- **Challenges:**

- **Data collection:** The set will consist of historical data on top 1000 open source projects with at least 50 stars. The data can be collected by using GitHub API.
- **Feature extraction:** GitHub API will return a number of activities and information about a repository e.g., commits, forks, watchers etc
- **Prediction technique:** Apply different prediction models and train the models.
- **Accuracy of prediction:** Use different evaluation methods to check the model's accuracy.



Project-3

- Tasks:
 - Design a scalable architecture for the framework
 - Develop a complete software solution based on your proposed architecture
- You can design different experiments to showcase the framework's features and model accuracy.
- Following are the recommended experiments:
 - **Platform performance**
 - **Predictive model accuracy**
- The implemented solution should have following features:
 - Automation
 - Contextualization
 - Orchestration



UPPSALA
UNIVERSITET

- Collaboration document
- Compulsory to submit with the report

Mini-projects

Group Contribution Document

Project:

Date: DD-MM-YYYY

Group name: AAC-100

Group members: AAA, BBB, CCC, DDD

Contributions

AAA: Read two articles "Article-1" and "Article-2", contributed on the framework design, work on overall framework integration and contribute in the report writing.

BBB: Implement first component of the framework, contributed on the framework design and participated in the report writing.

CCC: Implement third component of the framework, test the working of the framework, contribute in the report writing.

DDD: Read three articles "Article-1", "Article-2" and "Article-3", Implement the second component and contributed in report writing.

Signature and Date:

AAA

BBB

CCC

DDD



Data Engineering-II course

Learning Outcomes

On completion of the course the students shall be able to:

- **describe pros and cons of modern systems for handling data streams and use them in practice to address application needs;**
- **analyze properties of data intensive applications relying on streaming data and apply it to propose suitable solution architectures, including combination or batch and streaming data;**
- **implement software where the analysis from the previous point and technology addressed in the course is used;**
- **account for and handle practical aspects related to putting machine learning models into production;**
- **use frameworks for large-scale distributed machine learning;**
- **critically analyze, discuss and present solutions and implementations in writing and orally.**



Tips for getting started

Set up a start-up meeting and:

- Decide on “rules” for the group work. For example, how will you communicate, timelines etc.
- Make sure that you all have access to the Git/BitBucket repository.
- Agree on how to work with the code, is it OK to push to the master branch? Should you use feature branches, or will you work mainly with forks and Pull requests. Policies for code review?
- Assign shared responsibilities, is it possible to divide up some responsibilities on individuals, such as reading selected references and summarizing them to each other?



Writing the report

- Should be formatted as a short scientific paper
- Model it after a paper you have read during the course.
- Max 3000 words.
- Even though short, it should have all the typical parts of a scientific paper, including references to relevant literature.



Some thoughts

Project grading will be based on the defined project scope.

These are some interesting ideas that I want to share.

I encourage you to use AI tools to bring innovation to your projects!

For example:

- Can we use AI tools to build a Git crawler?
- Can we use free online AI tools to generate analytics?
- How can we effectively design a solution (aligned with the project scope) where online AI tools play a more instrumental role?



Grading Criteria

Quality of analysis and software development

To assess the quality of the proposed solution we will look at:

- How well does the taken approach fit the problem?
- How well can we expect the chosen approach to scale?
 - You need to demonstrate the scaling properties of your solution.
- We will look at the activity in GitHub or BitBucket, such as commit histories, issues/discussions, pull requests etc.
- How well is the code structured, annotated and usable by a third party?
 - Good solutions may not require a lot of code to be written. Attempt to use external libraries/tools whenever possible.
 - Is there some degree of automation? Is resources used statically or dynamically etc.?
 - How hard is it to use or extend the code/service?



Grading Criteria

Quality of the project paper

To assess the quality of the project paper, we will look at:

- How clearly is the problem stated?
- How well is the problem contextualized?
- How well does it cover related, relevant work?
 - This is supposed to be a short project, and we do not expect an extensive bibliography. We will look at how relevant the cited related work is and if it is accurately used to support the discussion, rather than the sheer amount of references. Aim at approximately 4-5 references. You may of course cite papers from the literature seminar if they are relevant.
- How well do you argue for you proposed solution?
- How well do you demonstrate the suitability of your solution?
 - Performance benchmarks? Weak scaling properties?
 - How well are limitations discussed?
- Language: Is the paper clearly written using correct English?



UPPSALA
UNIVERSITET

Introduction to version control and project management with Git

Use of a version control system is essential for all code development. There are nice, simple, popular web services for making that easy, and that add a lot of value in terms of project management. Very useful for own development, but the benefits are even more apparent when working in teams.

Have you worked with Subversion (SVN)?

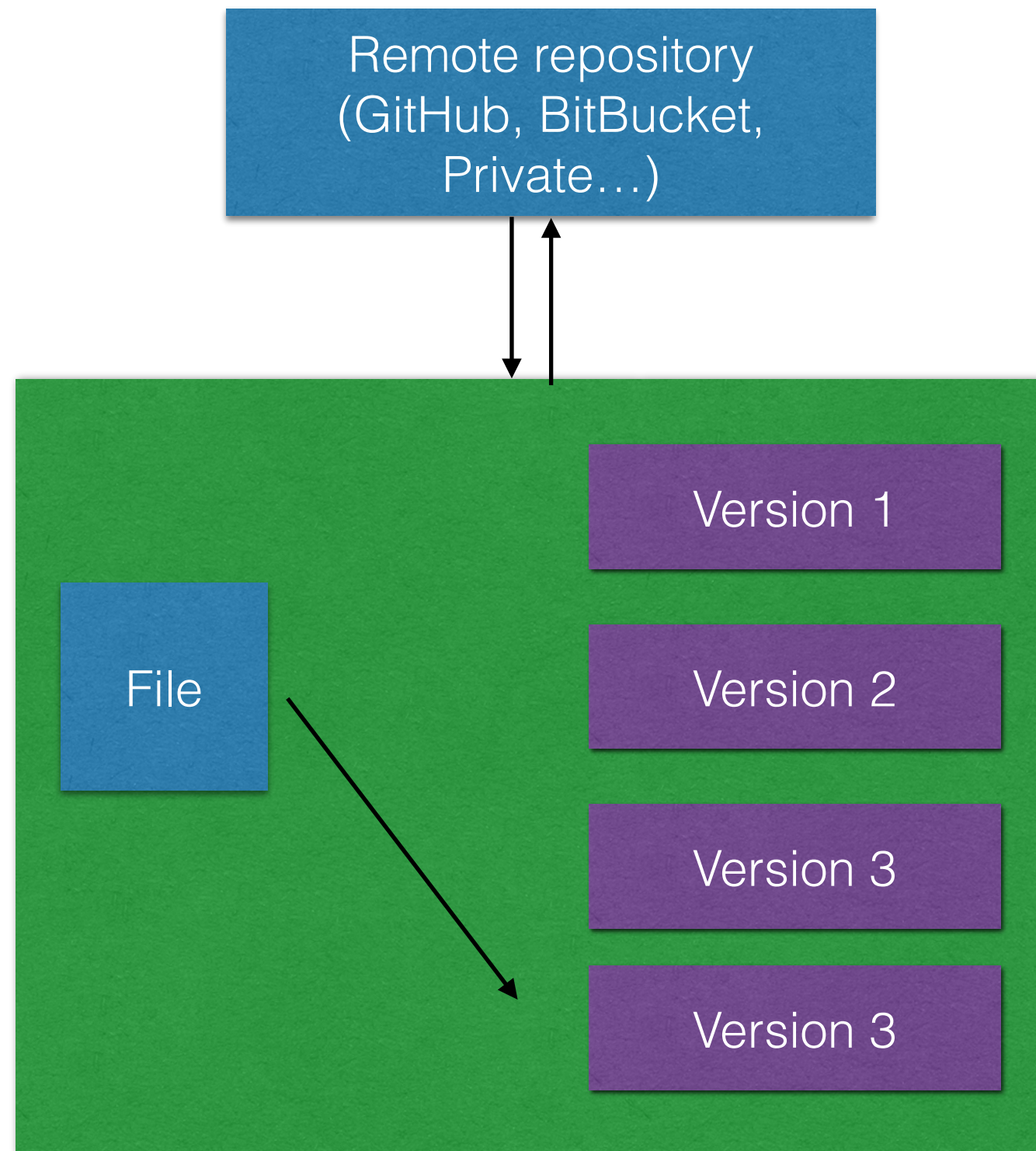
Then you will understand the basics. Git supports the typical centralized workflow often used together with SVN. But beware, there are important differences.

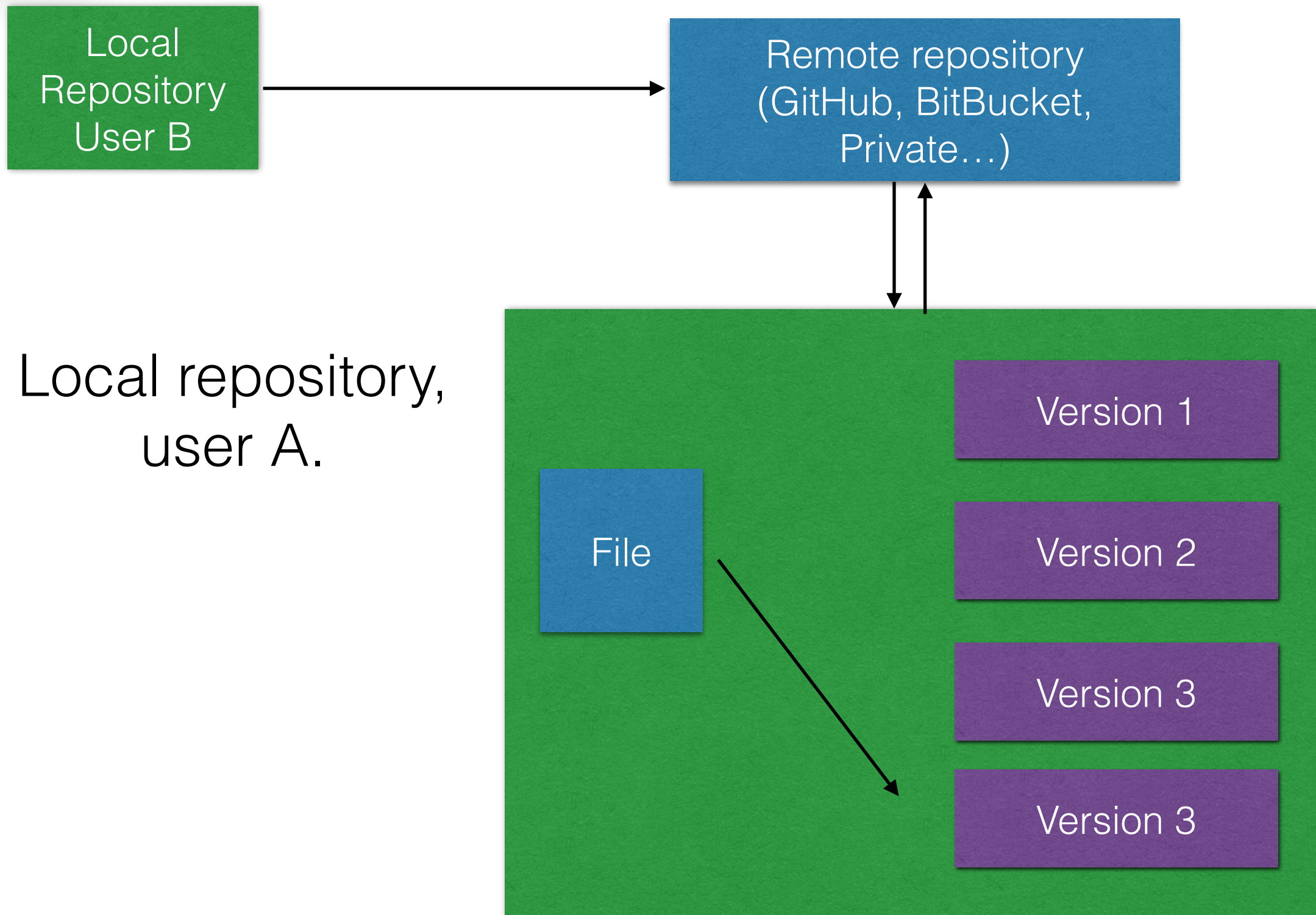


UPPSALA
UNIVERSITET

Git:

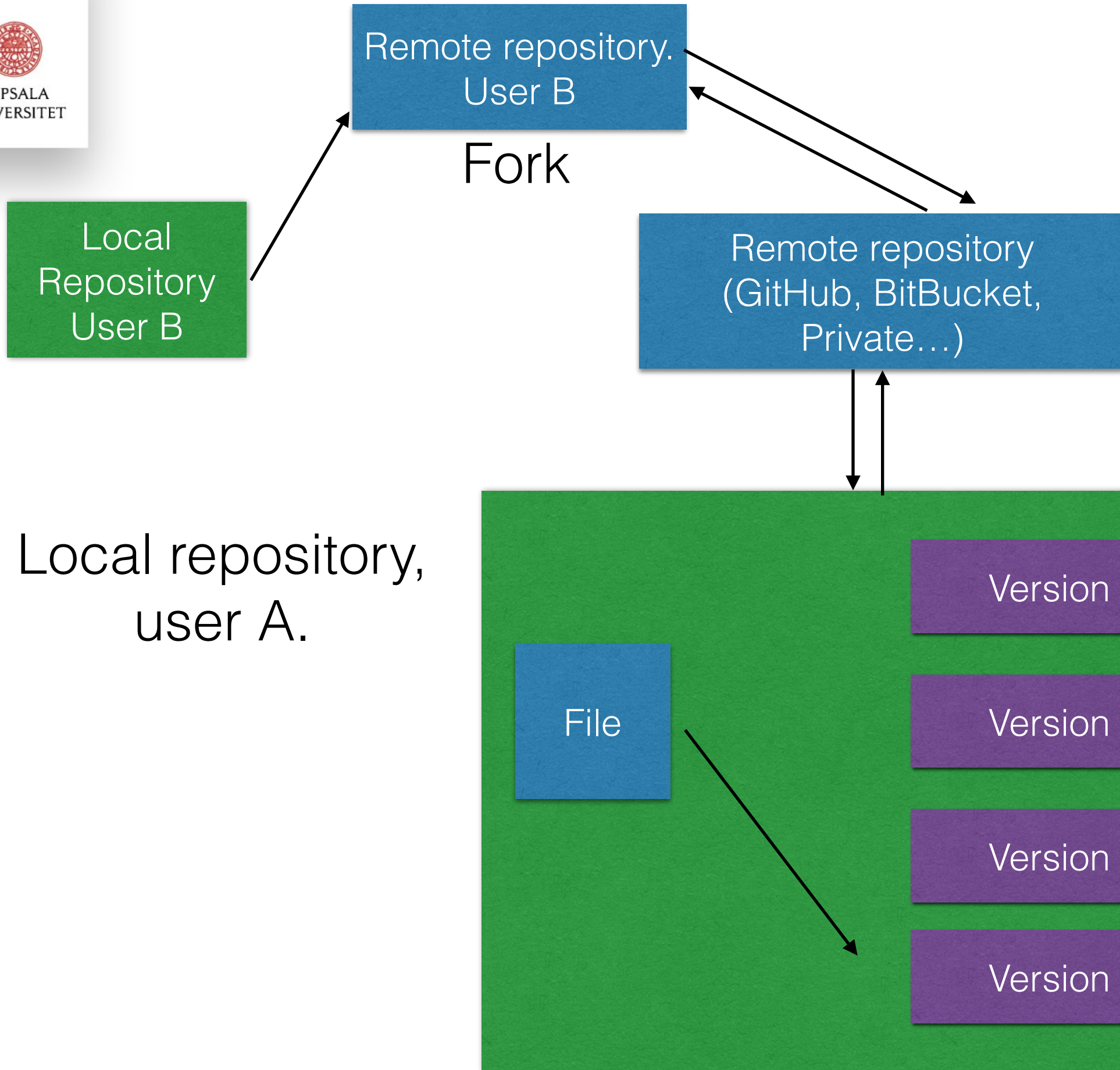
Local repository,
user A.

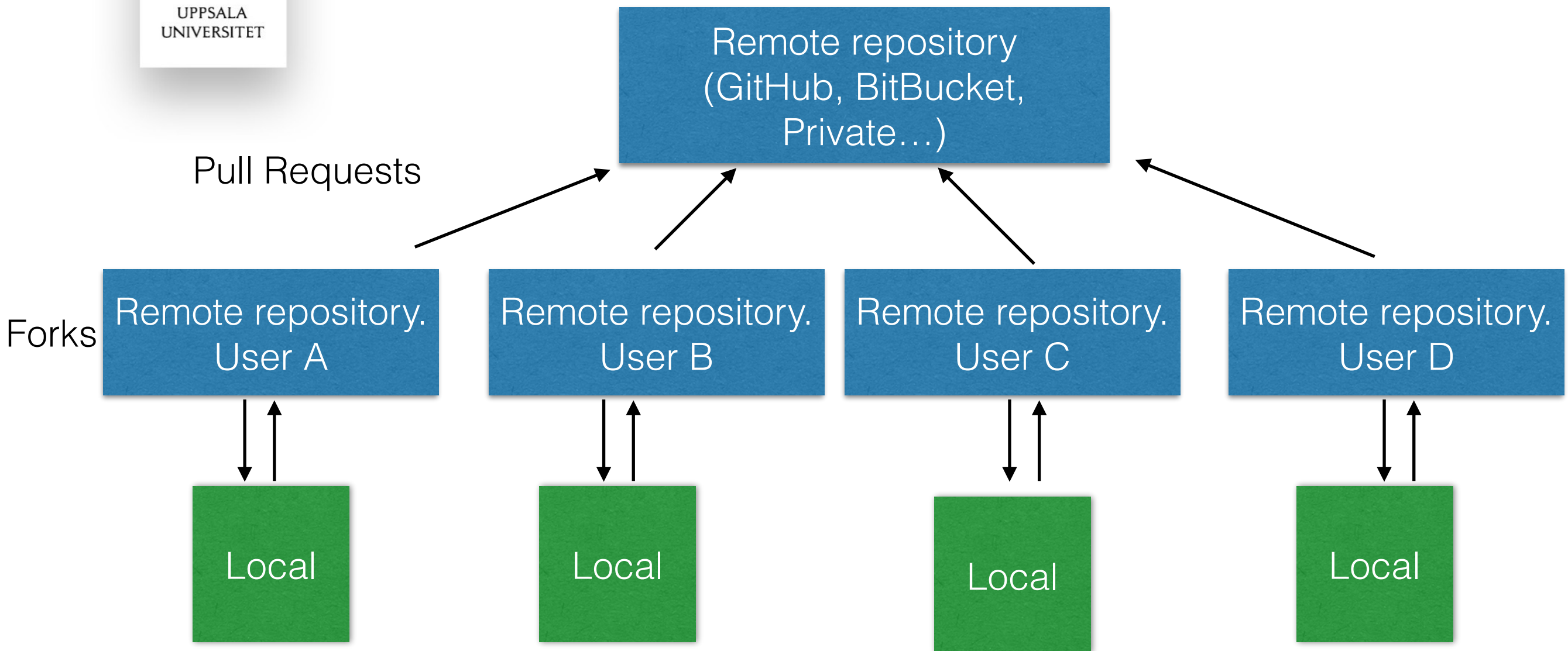






UPPSALA
UNIVERSITET





A fork is not a way to “steal code”, it is a convenient way to start contributing to a project that does not require granting write permissions to the original repository. It is a popular model in the OpenSource community.



“Practical Tips”...

Don't treat your VMs as your babies. Rather, treat it as another software component.

- Count on the VMs failing occasionally.
- If you have put in extensive work in manually installing software, consider using some contextualization tool to be able to automate the process.
- Consider the complexity in maintaining the environment when designing your solution, a complex stack takes more time to maintain.
- Keep critical data, such as your code, in external repositories at all times.
- Terminate unused VMs, they may put unnecessary strain on the system.
- Step by step serialize all the information collected from git API.
- Keep track of the logs while running the APIs.

Are now requirements of a good solution.